502094 - Software Deployment, Operations And Maintenance

Module 4: Cloud Infrastructure with AWS

Lecturer: MSc. Mai Van Manh

Email: maivanmanh@tdtu.edu.vn

Lecture Overview

- Introduce **cloud computing** and **AWS global infrastructure**
- Create an **AWS account** and understand the new **Free Plan / credits model**
- Use both **AWS Management Console** and **AWS CLI**
- Explore core services: **EC2, S3, RDS, Lambda**
- Deep dive into **EC2, key pairs, security groups, Elastic IP**
- Walk through **manual deployment** of Node.js & Spring Boot apps on EC2
- Discuss **cost awareness** and basic optimization strategies
- Preview **hands-on demos** and later modules (load balancing, auto scaling, HTTPS)

Section 1

Introduction to Cloud Computing & AWS Global Infrastructure

- What is **cloud computing**?
- Why cloud matters for **backend / DevOps** engineers
- AWS as a **leading cloud provider**
- Regions, Availability Zones, Edge locations
- Shared Responsibility Model (high-level)

What Is Cloud Computing?

- **Cloud computing** = on-demand delivery of IT resources (compute, storage, databases, etc.) over the internet
- You **rent** resources from a provider (AWS) instead of buying servers
- Key characteristics:
  - **On-demand self-service**
  - **Broad network access**
  - **Resource pooling**
  - **Rapid elasticity**
  - **Measured service (pay-as-you-go)**
- For DevOps: enables **fast provisioning**, experimentation, and **scaling**

Traditional vs Cloud Infrastructure

- Traditional on-premises:
  - Buy hardware, rack servers, configure networking
  - Plan capacity for **peak** load (often underutilized)
  - Slow to scale and difficult to experiment
- Cloud:
  - Provision instances in **minutes**
  - Scale **up/down** with demand
  - Pay only for **what you use**
- Example: spinning up a **test environment** for a new API in AWS instead of requesting a physical server

Cloud Service Models

- **IaaS (Infrastructure as a Service)**
  - Virtual machines, networking, storage (e.g., **EC2**, **EBS**)
- **PaaS (Platform as a Service)**
  - Managed runtime environments (e.g., **Elastic Beanstalk**, **Heroku**-like services)
- **SaaS (Software as a Service)**
  - Ready-to-use apps (e.g., **Gmail**, **Office 365**, **Salesforce**)
- DevOps primarily works at **IaaS / PaaS** layers for deployment and operations

AWS at a Glance

- **Amazon Web Services (AWS)** is one of the major cloud platforms
- Offers 150+ services: compute, storage, databases, analytics, AI/ML, networking, etc.
- Common benefits:
  - **Global reach** (regions around the world)
  - **Mature ecosystem** (docs, SDKs, communities)
  - Tight integration between services
- In this course we focus on a **core subset** needed for web applications

AWS Global Infrastructure

- AWS is organized into:
  - **Regions**: physical locations with multiple data centers (e.g., `ap-southeast-1` in Singapore)
  - **Availability Zones (AZs)**: multiple isolated data centers within a region
  - **Edge locations**: cached content & low-latency endpoints for **CloudFront** and other services
- You always choose a **region** when you create resources
- For Vietnam-based users, common regions:
  - `ap-southeast-1` (Singapore)
  - `ap-southeast-2` (Sydney)

Regions & Availability Zones – Why They Matter

- **Regions**:
  - Data residency (where data is stored)
  - Latency (distance to users)
  - Service availability (not all services are in every region)
- **Availability Zones**:
  - Fault isolation: if one AZ fails, others remain working
  - High availability architectures typically spread across **at least 2 AZs**
- Example: deploying an EC2 autoscaling group across **3 AZs** for a production web app

Edge Locations & Content Delivery

- **Edge locations** = globally distributed sites used by **Amazon CloudFront**
- Purpose:
  - Cache static content (images, JS, CSS) nearer to users
  - Reduce latency and offload traffic from origin servers
- For DevOps:
  - Configure **CDNs** for static assets
  - Combine with S3 for **static website hosting**

AWS Shared Responsibility Model (High-Level)

- AWS and customers **share responsibilities** for security and operations
- AWS is responsible **for the cloud**:
  - Physical infrastructure, networking, hypervisor, managed services
- You are responsible **in the cloud**:
  - OS configuration, application code, data protection, IAM configuration
- For DevOps:
  - Must design secure **EC2** instances, **security groups**, **IAM roles**, and **backup strategies**

Section 2

Creating an AWS Account & Understanding Free Plan

- Steps to register a new AWS account
- The new **credit-based Free Plan** (after July 2025)
- Essential security steps after creation

Creating an AWS Account – Overview

- Visit **https://aws.amazon.com/**
- Click **Create an AWS Account**
- Provide:
  - Email, password, account name
  - Contact information (personal or business)
  - Credit/debit card (for identity verification)
  - Phone verification (SMS or call)
- Choose between **Free plan** and **Paid plan**
- After activation, you can sign in to the **AWS Management Console**

AWS Free Plan (After July 2025)

- For new customers (accounts created after **July 15, 2025**):
  - Receive **$100 sign-up credit**
  - Can earn up to **$100 additional credits** by completing learning activities
  - Valid for **up to 6 months** or until credits are fully used.
- Access to **30+ “always free” services** with usage limits
- Two plan types:
  - **Free account plan**: no charges, but limited services and auto-close after credits / duration
  - **Paid account plan**: full service access, pay-as-you-go beyond credits

Free Plan vs Paid Plan – For Students

- **Free account plan**:
  - Ideal for **learning and experiments**
  - No risk of unexpected billing (account auto-closes)
  - Cannot access some advanced services (e.g., Reserved Instances)
- **Paid account plan**:
  - For serious **projects or startups**
  - Enables advanced features and all services
  - Must actively manage costs
- In this course, we usually start with **Free account plan** for labs and demos

Essential Security Steps After Account Creation

- **Do NOT** use the root account for day-to-day activities
- Immediately:
  - Enable **MFA** (Multi-Factor Authentication) on root
  - Create an **IAM admin user** (or group) with administrative permissions
  - Use **strong passwords** and **password manager**
- Set up **billing alerts** and **budgets** to avoid surprises

Creating an IAM Administrative User

- Sign in as **root user** once
- Open **IAM** (Identity and Access Management)
- Steps:
  - Create an **IAM group** (e.g., `Administrators`) with `AdministratorAccess` policy
  - Create a **new IAM user** (e.g., `devops-admin`)
  - Assign user to `Administrators` group
  - Enable **MFA** for this IAM user
- From now on, log in using the **IAM user** for console work

Section 3

AWS Console & AWS CLI

- Navigate the **AWS Management Console**
- Install and configure **AWS CLI**
- Use CLI for basic tasks (EC2, S3)

AWS Management Console Tour

- Web-based GUI at **https://console.aws.amazon.com/**
- Key components:
  - **Region selector** (top-right)
  - **Search bar** for services (e.g., type `EC2`, `S3`)
  - **Service-specific dashboards**
- For new users:
  - Pin frequently used services to the **Favorites** bar
  - Always check the **selected region** before creating resources

Installing AWS CLI (macOS, Linux)

- Preferred installation (macOS, Linux):
  - Using a bundled installer
- Example for **Linux (Ubuntu)**:

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install
aws --version
```

- Example for **macOS (pkg)**:
  - Download installer from AWS website and run the `.pkg` file
  - Verify:

```
aws --version
```

Installing AWS CLI (Windows)

- Download MSI installer from AWS website
- Run installer with default options
- Verify in **PowerShell**:

```
aws --version
```

- Ensure the `aws` command is added to the **PATH** environment variable

Configuring AWS CLI with `aws configure`

- Use IAM user **Access key ID** and **Secret access key**
- In terminal:

```
aws configure
AWS Access Key ID [None]: AKIA...
AWS Secret Access Key [None]: ********************
Default region name [None]: ap-southeast-1
Default output format [None]: json
```

- This creates a config file in `~/.aws/credentials` and `~/.aws/config`
- Use **least privilege** for production IAM users (not full admin)

Verifying CLI Connectivity

- List all S3 buckets:

```
aws s3 ls
```

- List EC2 instances in current region:

```
aws ec2 describe-instances --query "Reservations[*].Instances[*].InstanceId" --output table
```

- If credentials or region are incorrect, the CLI will report errors
- For DevOps: CLI is essential for **scripting, automation, and CI/CD**

Using Multiple AWS Profiles

- Configure additional profiles with:

```
aws configure --profile tdtu-student
```

- Use a specific profile:

```
aws s3 ls --profile tdtu-student
```

- Good practice:
  - Separate **personal**, **university**, and **work** accounts
  - Avoid mixing **production** and **sandbox** environments

Section 4

Overview of Common AWS Services

- **EC2** – virtual servers
- **S3** – object storage
- **RDS** – managed relational databases
- **Lambda** – serverless compute

Amazon EC2 – Elastic Compute Cloud

- **EC2** provides resizable **virtual machines** in the cloud
- You choose:
  - **Instance type** (CPU, RAM)
  - **Storage** (EBS volume size and type)
  - **Network** (VPC, subnet, security groups)
- Use cases:
  - Web servers, API servers
  - Background jobs, batch processing
  - Custom runtimes

Amazon S3 – Simple Storage Service

- **Object storage** for any type of file: images, backups, logs, static site files
- Key properties:
  - Data stored in **buckets**
  - High durability (designed for **11 nines**)
  - Fine-grained access control (bucket policies, IAM)
- Use cases:
  - Static website hosting
  - Application assets (images, CSS, JS)
  - Central log storage and backups

Amazon RDS – Relational Database Service

- Managed relational databases:
  - **MySQL, PostgreSQL, MariaDB, Oracle, SQL Server, Aurora**
- AWS handles:
  - Patching, backups, multi-AZ failover (if enabled)
- You focus on:
  - Schema design, queries, performance tuning
- Use cases:
  - Application databases for web and mobile apps

AWS Lambda – Serverless Functions

- **Function-as-a-Service (FaaS)**:
  - Upload code; AWS handles servers, scaling, and availability
- Execution model:
  - Triggered by events (HTTP via API Gateway, S3 uploads, SQS messages, etc.)
- Benefits:
  - No server management
  - Automatic scaling based on demand
- Use cases:
  - Lightweight APIs, cron jobs, event processing

Choosing Between EC2, RDS, Lambda, S3

- Web backends:
  - Start with **EC2 + RDS + S3**
- Short-running background tasks:
  - Consider **Lambda**
- Static assets:
  - Store in **S3** + **CloudFront**
- As DevOps engineers, you must understand **trade-offs** in cost, scalability, and operational overhead

Section 5

Deep Dive into Amazon EC2

- Instance types & families
- AMIs
- Key pairs
- Security groups
- Elastic IPs
- Preview: Load balancers & Auto Scaling

EC2 Instance Types & Families

- EC2 instances are grouped into **families**:
  - **General Purpose** (e.g., `t3`, `t4g`, `m6g`) – balanced CPU/RAM
  - **Compute Optimized** (e.g., `c6g`) – CPU-heavy workloads
  - **Memory Optimized** (e.g., `r6g`) – in-memory DBs, caching
  - **Storage / I/O Optimized** (e.g., `i3`, `i4`) – heavy disk I/O
- For student projects:
  - Use small general-purpose instances like `t3.micro` or `t3.small`

Choosing an Instance Type – Example

- Example: Deploying a Node.js API
  - Requirements: low to moderate CPU, 1–2 GB RAM
- Candidate types:
  - `t3.micro` – 2 vCPU (burstable), 1 GB RAM
  - `t3.small` – 2 vCPU, 2 GB RAM
- Start small for **development**, monitor usage with **CloudWatch**, then scale up if needed

Amazon Machine Images (AMIs)

- An **AMI** is a template used to launch EC2 instances:
  - OS (Ubuntu, Amazon Linux, Windows, etc.)
  - Pre-installed software
- Types:
  - **AWS-managed public AMIs**
  - **Marketplace AMIs** (partner-provided)
  - **Custom AMIs** (your own image with pre-configured software)
- For this module:
  - We will use **Ubuntu Server LTS** AMIs

Selecting AMIs – Example

- From EC2 console:
  - Choose **Launch instance**
  - Search for **Ubuntu Server 22.04 LTS**
- AWS CLI equivalent:

```
aws ec2 describe-images \
--owners 099720109477 \
--filters "Name=name,Values=ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*" \
--query "Images[0].ImageId"
```

- Owner `099720109477` is Canonical (Ubuntu) official account

Key Pairs for SSH Authentication

- **Key pair** = public key + private key used for SSH login
- You create or upload a **public key** to AWS; AWS stores it
- When launching an instance, you select a key pair
- Connect from your machine with the **private key** (`.pem` or `.ppk`)
- Never share your **private key**; protect it with correct file permissions

Creating and Using SSH Keys (Linux/macOS)

- Generate key pair locally:

```
ssh-keygen -t ed25519 -C "tdtu-devops-2025" -f ~/.ssh/tdtu-devops-2025
```

- Upload the public key to AWS (console or CLI):

```
aws ec2 import-key-pair \
--key-name "tdtu-devops-2025" \
--public-key-material fileb://~/.ssh/tdtu-devops-2025.pub
```

- Use the private key to connect:

```
ssh -i ~/.ssh/tdtu-devops-2025 ubuntu@EC2_PUBLIC_IP
```

Security Groups – Virtual Firewalls

- **Security groups** control inbound and outbound traffic to your EC2 instances
- They are **stateful**:
  - If inbound is allowed, response traffic is automatically allowed
- Best practices:
  - Allow **SSH (22)** only from your IP
  - Allow **HTTP (80)** and **HTTPS (443)** from **anywhere** for public web servers
  - Avoid `0.0.0.0/0` for non-web ports like DB (3306)

Example: Web Server Security Group

- Inbound rules:
  - SSH: `22` from `YOUR_PUBLIC_IP/32`
  - HTTP: `80` from `0.0.0.0/0`
  - HTTPS: `443` from `0.0.0.0/0`
  - Optional app ports (e.g., `3000`, `8080`) from `0.0.0.0/0` for testing
- Outbound rules:
  - Allow all traffic (default) – typically fine for web servers
- Constantly review rules to enforce **least privilege**

Elastic IP Addresses (EIP)

- **Elastic IP** = static public IPv4 address you can attach to EC2
- Benefits:
  - IP remains the same even if you stop/start or replace the instance
- Important cost notes:
  - EIP is free only when **associated with a running instance**
  - Unused EIPs are charged
- Typical pattern:
  - Allocate one EIP for your **production** server or load balancer

Preview: Load Balancers & Auto Scaling

- AWS offers **Elastic Load Balancing (ELB)**:
  - Distribute traffic across multiple instances
  - Health checks and automatic failover
- **Auto Scaling Groups (ASG)**:
  - Automatically add/remove instances based on metrics (CPU, requests)
- In this module we **preview** the concepts; a later module explores them deeply

Section 6

Deploying Applications on EC2 (Manual Workflow)

- Launch Ubuntu instances
- Connect via SSH
- Install runtimes and deploy:
  - Node.js + PM2
  - Spring Boot (Java)

Launching an Ubuntu EC2 Instance (Console)

- Steps:
  i. In EC2 console, choose **Launch instance**
  ii. Enter name: `devops-node-lab`
  iii. Select AMI: **Ubuntu Server 22.04 LTS**
  iv. Select instance type: `t3.micro`
  v. Configure key pair: `tdtu-devops-2025`
  vi. Configure security group (ports 22, 80, 3000)
  vii. Launch instance
- After a few minutes, note the **public IPv4 address**

Launching an EC2 Instance (CLI)

- Equivalent using AWS CLI:

```
aws ec2 run-instances \
--image-id ami-xxxxxxxxxxxxxxxxx \
--instance-type t3.micro \
--key-name tdtu-devops-2025 \
--security-group-ids sg-xxxxxxxxxxxxxxxxx \
--subnet-id subnet-xxxxxxxxxxxxxxxxx \
--tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=devops-node-lab}]'
```

- Replace `ami-*`, `sg-*`, `subnet-*` with actual values from your account
- Use `aws ec2 describe-instances` to find the public IP

SSH into Ubuntu EC2 Instance

- From local machine:

```
ssh -i ~/.ssh/tdtu-devops-2025 ubuntu@3.12.34.56
```

- First login:
  - Check OS version:

    ```
    lsb_release -a
    ```

  - Update package index:

    ```
    sudo apt update && sudo apt upgrade -y
    ```

- Use `tmux` or `screen` to avoid losing session during long tasks

Installing Node.js & PM2 (Production Setup)

- Install Node.js LTS (via NodeSource):

```
curl -fsSL https://deb.nodesource.com/setup_lts.x | sudo -E bash -
sudo apt install -y nodejs
node -v
```

- Install **PM2** globally:

```
sudo npm install -g pm2
```

- Clone your app and install dependencies:

```
git clone https://github.com/tdtu-devops/sample-node-api.git
cd sample-node-api
npm install
```

Running Node.js App with PM2

- Start the app:

```
pm2 start app.js --name sample-node-api
pm2 status
```

- Enable PM2 startup on reboot:

```
pm2 startup systemd
pm2 save
```

- Verify from browser:
  - Visit `http://EC2_PUBLIC_IP:3000/`
- For production, front the app with **Nginx** reverse proxy on port 80

Installing Java & Deploying Spring Boot

- Install OpenJDK (e.g., Java 17):

```
sudo apt update
sudo apt install -y openjdk-17-jre
java -version
```

- Copy Spring Boot jar (via `scp`):

```
scp \
-i ~/.ssh/tdtu-devops-2025 \
target/demo-0.0.1-SNAPSHOT.jar \
ubuntu@EC2_PUBLIC_IP:~/apps/demo.jar
```

- Run the app:

```
java -jar ~/apps/demo.jar --server.port=8080
```

Running Spring Boot as a Systemd Service

- Create service file `/etc/systemd/system/demo.service`:

```
[Unit]
Description=Spring Boot Demo Service
After=network.target

[Service]
User=ubuntu
ExecStart=/usr/bin/java -jar /home/ubuntu/apps/demo.jar
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

- Enable and start service:

```
sudo systemctl daemon-reload
sudo systemctl enable demo
```

Logs and Service Lifecycle

- PM2 logs:

```
pm2 logs sample-node-api
```

- Systemd logs for Spring Boot:

```
journalctl -u demo -f
```

- Understanding logs is crucial for:
  - Debugging deployment issues
  - Monitoring performance
  - Incident response

Section 7

Cost Awareness & Basic Optimization

- Pricing models: On-Demand, Reserved, Spot
- Free plan usage & pitfalls
- Stopping vs terminating instances
- Storage (EBS, S3) cost awareness
- Monitoring with Billing & CloudWatch

AWS Pricing Models

- **On-Demand instances**:
  - Pay per hour/second, no long-term commitment
  - Flexible but more expensive
- **Reserved Instances / Savings Plans**:
  - Commit to 1 or 3 years
  - Lower hourly price
- **Spot Instances**:
  - Use spare capacity at significant discount
  - Can be interrupted at any time
- For students: mainly use **On-Demand** on small instances

Free Plan & Common Pitfalls

- Free plan credits can be used on many services, but:
  - Some services are **excluded**
  - Credits expire after **6 months**
- Typical traps:
  - Forgetting to stop or terminate EC2 instances
  - Leaving large **EBS volumes** or **snapshots**
  - Storing too much data in **S3** or enabling high-traffic distributions
- Always review the **Billing dashboard**

Stopping vs Terminating Instances

- **Stopping**:
  - Instance is shut down
  - You stop paying for **compute**
  - You still pay for **EBS storage**
  - You can start again with same root disk
- **Terminating**:
  - Instance and (optionally) EBS volumes are deleted
  - No further charges for that instance
  - Irreversible (unless you have snapshots)
- For labs: terminate instances you do not need

Storage Cost Awareness (EBS, S3)

- **EBS (Elastic Block Store)**:
  - Charged per GB-month of provisioned storage
  - Also pay for **snapshots**
- **S3**:
  - Charged per GB-month of stored data + requests + data transfer
- Cost-saving tips:
  - Delete unused EBS volumes and snapshots
  - Clean up old log files in S3
  - Use **lifecycle policies** to move cold data to cheaper tiers

Monitoring Billing & Usage

- Open **Billing & Cost Management** console:
  - Set up **budgets** and **alerts** (e.g., notify at $5, $10)
  - Review **Cost Explorer** for service-level breakdown
- Use **CloudWatch** to monitor:
  - CPU, network, disk metrics
  - Custom metrics (e.g., request rate)
- For student projects: check billing at least **once per week**

Section 8

Hands-On Demonstrations

- Static website hosting with S3
- Launching & managing EC2 servers
- Deploying Node.js & Spring Boot apps
- Optional: Domain, DNS, HTTPS

S3 Static Website – Concept

- Host static content (HTML, CSS, JS) directly from **S3**
- No servers to manage
- Combine with **CloudFront** for global distribution
- Typical use cases:
  - Landing pages
  - Single Page Applications (SPAs)
  - Documentation sites

S3 Static Website – Setup Example

- Create bucket `tdtu-devops-2025-site` in `ap-southeast-1`
- Enable **static website hosting** and upload `index.html`

```html
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8" />
  <title>TDTU DevOps Demo</title>
</head>
<body>
  <h1>Hello from S3 Static Website!</h1>
  <p>Deployed as part of Module 4 — Cloud Infrastructure with AWS.</p>
</body>
</html>
```

- Configure bucket policy to allow public read for website objects (for demo only)

Generating and Uploading SSH Keys for EC2

- On your laptop:

```
ssh-keygen -t ed25519 -f ~/.ssh/tdtu-aws-lab -C "tdtu-lab"
```

- Upload key to AWS:

```
aws ec2 import-key-pair \
--key-name "tdtu-aws-lab" \
--public-key-material fileb://~/.ssh/tdtu-aws-lab.pub
```

- When launching instance, choose `tdtu-aws-lab` as **key pair**

Creating Security Groups for the Lab

- Lab security group example (name: `devops-lab-sg`):
  - Inbound:
    - 22 (SSH) from your IP
    - 80 (HTTP) from `0.0.0.0/0`
    - 443 (HTTPS) from `0.0.0.0/0`
    - 3000, 8080, 3306 from `0.0.0.0/0` for lab testing
- In production:
  - Restrict DB port 3306 to **app servers only**
  - Use **bastion hosts** or **VPN** for SSH

Launching Ubuntu EC2 Instances for the Lab

- Use console or CLI to launch:
  - Instance `devops-node-lab` (Node.js)
  - Instance `devops-spring-lab` (Spring Boot) – optionally on same or separate instance
- Attach:
  - Key pair: `tdtu-aws-lab`
  - Security group: `devops-lab-sg`
- Verify you can SSH into both instances

Deploying Node.js App – Lab Steps

- On the instance:

```
sudo apt update
curl -fsSL https://deb.nodesource.com/setup_lts.x | sudo -E bash -
sudo apt install -y nodejs git
git clone https://github.com/tdtu-devops/sample-node-api.git
cd sample-node-api
npm install
npm run build || echo "skip if not using build"
```

- Run using PM2 and expose on port 3000
- Confirm from browser: `http://EC2_PUBLIC_IP:3000/`

Deploying Spring Boot App – Lab Steps

- On the instance:

```
sudo apt update
sudo apt install -y openjdk-17-jre
mkdir -p ~/apps
```

- Upload `demo.jar` using `scp` or GitHub release
- Create and enable `demo.service` via systemd (as shown earlier)
- Ensure app listens on port 8080, test via:

```
curl http://localhost:8080/
```

Optional: Registering a Domain & Route 53

- Register a domain (e.g., via Route 53 or external registrar)
- In Route 53:
  - Create a **Hosted Zone** for your domain (e.g., `devops2025.tdtu.vn`)
  - Add **A record** pointing to your EC2 **Elastic IP**
- After DNS propagation:
  - Access your app via `https://api.devops2025.tdtu.vn/`

Optional: Installing HTTPS with Caddy

- Install **Caddy** (auto HTTPS reverse proxy):

```
sudo apt install -y debian-keyring debian-archive-keyring apt-transport-https
curl https://dl.caddyserver.com/api/download?os=linux&arch=amd64 | sudo bash
```

- Example Caddyfile `/etc/caddy/Caddyfile`:

```
api.devops2025.tdtu.vn {
  reverse_proxy localhost:3000
}
```

- Caddy automatically obtains and renews **Let's Encrypt** certificates

Optional: Installing HTTPS with Nginx + Certbot

- Install Nginx:

```
sudo apt update
sudo apt install -y nginx
```

- Configure a server block for your domain
- Install Certbot:

```
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d api.devops2025.tdtu.vn
```

- Certbot:
  - Obtains SSL certificates
  - Configures Nginx for HTTPS
  - Sets up automatic renewals

Summary & Next Steps

- You learned:
  - Core cloud computing concepts and AWS global infrastructure
  - How to create a secure AWS account and use the Free Plan
  - Basics of AWS Console and CLI
  - Overview of EC2, S3, RDS, Lambda
  - Manual deployment of Node.js and Spring Boot apps on EC2
  - Fundamental cost awareness and best practices